

A Middleware-Centric Architectural Digital Model for Pre-Deployment Verification and CI/CD Gating in Naval Combat Management Systems (Industry Track)

Ibrahim Onuralp Yigit

Command Control and Defense Technologies, HAVELSAN
Inc.
Istanbul, Turkiye
iyigit@havelsan.com.tr

Industrial Co-Authors & CMS Architects
Naval Combat Systems Directorate, HAVELSAN Inc.
Ankara, Turkiye
contact@havelsan.com.tr

Abstract

Modern mission-critical Naval Combat Management Systems (CMS), such as HAVELSAN's combat-proven ADVENT CMS, are distributed real-time systems built on publish/subscribe middleware, integrating hundreds of applications across operator consoles, radar trackers, weapon controllers, and tactical data links. Their most severe integration defects are non-local architectural misconfigurations: incompatible Quality-of-Service (QoS) contracts on hard-deadline weapon assignment channels, overlapping CPU core affinity masks, orphaned tactical topics, and circular package dependencies. Such defects survive unit and component testing and surface only during shipyard integration or sea trials.

We report on *System as a Graph* (SaaG), an architectural digital model for pre-deployment verification and CI/CD gating. Reconstructed from candidate release descriptors rather than run-time synchronization, SaaG builds an attributed typed multigraph ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$) over applications, brokers, topics, processing nodes, and shared libraries; derives asymmetric failure-dependency projections from pub/sub topology ($B \xrightarrow{\text{DEPENDS_ON}} A$); and gates the pipeline on static rule violations ranked by a severity rubric adapted from MIL-STD-882E.

Our central industrial finding is clear: graph analysis is not the computational bottleneck. Across five scaling operational naval platform profiles (1,498 to 3,254 components across up to 66 distributed nodes), static verification executes in 0.125 s on a single-node airborne profile and 1.152 s on the 66-node flagship profile (P95 1.161 s), representing under 0.22% of a typical 9-minute automated build pipeline. The rule catalog targets pub/sub misconfiguration classes that would otherwise surface only on physical testbeds. However, an industrial gap analysis reveals that of seven verification capabilities specified with naval combat system architects, six remain constrained by configuration-data acquisition across defense engineering silos rather than by algorithmic limits. We report the framework architecture, multi-scale verification latency, the misconfiguration classes the catalog targets, and practical lessons from building the prototype inside a defense engineering organization.

CCS Concepts

• **Computer systems organization** → **Reliability**; • **Software and its engineering** → **Software defect analysis**.

Keywords

Naval Combat Management Systems (CMS), ADVENT CMS, SaaG Framework, Architectural Digital Model, Middleware Verification, Pub/Sub QoS Contracts, CI/CD Gating, CPU Core Pinning, Static Rule Auditing, OMG DDS, MIL-STD-882E, Network Enabled Capability (NEC)

1 Introduction & Industrial Problem Statement

Continuous Integration and Continuous Delivery (CI/CD) pipelines have fundamentally transformed software engineering by automating compilation, testing, and artifact generation [15]. In naval defense systems, modern Combat Management Systems (CMS) have evolved from monolithic mainframes into modular, distributed, open-architecture systems governed by high-performance publish/subscribe (pub/sub) middleware [6].

1.1 The Pre-Deployment Verification Gap in Naval Combat Systems

Modern naval operations require rapid sensor-to-shooter loops, multi-sensor data fusion, coordinated weapon assignment, and seamless cross-platform interoperability. Built to fulfill these demands, **ADVENT CMS** (developed by **HAVELSAN** in cooperation with the Turkish Naval Forces Research Center Command—**ARMERKOM** [1]) represents a state-of-the-art naval combat suite spanning a wide product tree: **ADVENT Kalyon** and **Milgem CMS** for surface combatants (frigates, corvettes, MCM vessels) handling multi-threat warfare (AAW, ASuW, ASW, EW); **LHD CMS** for amphibious assault flagships (e.g., L400 TCG Anadolu), providing task group command, multi-link gateway centers, and unmanned vehicle control; **ADVENT Rota**, a mission management system for USV/UUV/UAV platforms with STANAG 4586 compliance [3]; **ADVENT Marti**, an airborne C2 system for maritime patrol aircraft and naval helicopters; and **ADVENT Ufuk**, a land-based coastal surveillance and C2 system compiling the Recognized Maritime Picture.

Across these platforms, ADVENT CMS comprises over **155 external system integrations**, **750 distributed software applications**, and **13,000,000 lines of code**. Subsystems communicate over **Genieware**, a national publish/subscribe middleware for real-time mission-critical defense systems. Genieware adopts the OMG DDS [6] data-centric model and its Request/Offered (RxO) contract semantics for the **RELIABILITY**, **DURABILITY**, **PARTITION**, and **DOMAIN_ID** policies—precisely the policies our rules evaluate—so

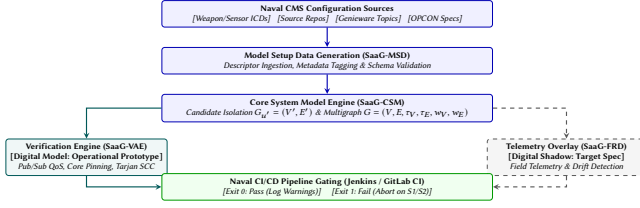


Figure 1: SaaG pipeline integration for ADVENT CMS. Dashed boxes are specified, not yet built.

we use DDS terminology throughout and the rules of §3 apply unchanged to DDS deployments. Genieware diverges in its discovery and wire protocol, which our static model does not represent. Platforms also interoperate over Tactical Data Links (TDLs: Link 11, Link 16 [4], Link 22 [5], and ADVENT Native Link H). Under the **Network Enabled Capability (NEC)** paradigm, ADVENT shares sensor tracks and weapon allocations across platforms, enabling Force-Wide Weapon/Sensor Allocation (WASA) and remote firing authorizations.

When a candidate software build is submitted for release into an operational naval platform, conventional CI/CD pipelines evaluate unit and module tests in isolation, so non-local architectural misconfigurations slip through. Latency-critical track fusion daemons or fire control calculators are pinned to CPU core masks overlapping non-real-time GUI renderers or loggers on the same multi-core console. Endpoints are bound with incompatible Request/Offered (RxO) QoS contracts—a weapon assignment subscriber requesting TRANSIENT_LOCAL durability against a fire control publisher offering only VOLATILE—so the endpoints never match and data never flows; the middleware raises an incompatible-QoS status, but operational code rarely traps it, making the failure effectively silent. Topic schemas diverge across platform releases, or refactored TDL forwarding topics are left with zero subscribers. And transitive dependency cycles between weapon allocation planners and threat evaluation modules surface as initialization deadlocks during OPCON start-up.

Provisioning full target hardware testbeds (physical multi-console Combat Information Centers [CIC], real sensor/weapon simulators, and multi-ship test ranges) for every candidate release is prohibitively expensive, logistically complex, and slow. Leaving architectural defects to be discovered during shipyard commissioning, harbor acceptance tests (HAT), or sea acceptance tests (SAT) leads to massive project delays and severe safety risks.

1.2 The Architectural Digital Model Paradigm & Dual Operational Usages

To eliminate this pre-deployment gap and manage system complexity across the lifecycle, we developed **System as a Graph (SaaG)**. Rather than relying on heavyweight runtime testbeds, SaaG constructs an attributed graph model of the target combat system topology directly from configuration artifacts and interface contracts.

The digital twin classification of Kritzinger et al. [7] distinguishes a **digital model** (no automated exchange with the physical system; built from static artifacts), a **digital shadow** (automated one-way

flow from the physical system to the model), and a **digital twin** (automated bidirectional synchronization). Under this taxonomy our framework is strictly an *architectural digital model*—reconstructed deterministically from configuration artifacts. In naval engineering practice, the architectural digital model serves two complementary operational usages: (1) **Pre-Deployment Verification & CI/CD Gating**, preventing non-conforming builds ($G_{M'}$) from reaching integration testbeds; and (2) **Architectural Improvement & Refactoring of Existing Platforms**, enabling architects to detect circular dependency cycles, discover criticality bottlenecks, prune dead topic configurations, and perform “what-if” impact simulations across fielded combat systems.

1.3 Key Industrial Contributions & Empirical Findings

This paper makes the following contributions:

- (1) **Naval CMS Architectural Model Baseline (§2):** A formal directed multigraph representation ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$) capturing 5 entity classes, 6 structural relations, and 6 failure-dependency projection rules that explicitly map the downstream propagation of architectural risk in pub/sub naval combat systems.
- (2) **CI/CD Pipeline Gating & Verification Methodology (§3, §4):** A two-stage deployment gate operating on standard CI runners with a defined safety severity rubric (S1–S4) adapted from MIL-STD-882E [2]. We characterize the mission-critical pub/sub misconfiguration classes the rule catalog targets (QoS mismatches, CPU core contention, topic continuity breaks, circular dependencies) and state which are covered by rules implemented in the prototype.
- (3) **Verification Complexity & Industrial Bottleneck Analysis (§5, §6):** Benchmarks across 5 naval platform profiles (1,498–3,254 components; 1–66 nodes) modeled on ADVENT platform lines show gate latency of 0.125–1.152 s, under 0.22% of a 9-minute build, with node distribution rather than component count driving cost. We document that 6 of 7 specified verification capabilities are blocked by configuration data acquisition across defense engineering silos, not by algorithmic complexity.

2 The Architectural Digital Model: Formulation and Graph Derivation

SaaG adopts and operationalizes the formal graph modeling foundation established by Yigit and Buzluca [17] for distributed publish-subscribe systems. We formalize the distributed naval CMS middleware topology as an attributed, weighted directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

2.1 Entity Classes and Structural Relations

The vertex set V is partitioned into five distinct entity types ($\tau_V : V \rightarrow \mathcal{T}_v$):

- **Applications (V_{app}):** Executable CMS software binaries.
- **Brokers (V_{broker}):** Broker services facilitating CMS communication.

- **Topics (V_{topic}):** Typed publish/subscribe communication channels carrying data payloads.
- **Infrastructure Nodes (V_{node}):** Physical OPCON console workstations, ruggedized VME/VPX server chassis, and mission computers.
- **Libraries (V_{lib}):** Shared dynamic libraries.

Six structural edge types ($\tau_E : E_{\text{structural}} \rightarrow \mathcal{T}_E$) are extracted directly from configuration descriptors: PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, and USES. We additionally write COLOCATED_WITH for the symmetric relation *induced* by two units sharing a RUNS_ON target; it is derived rather than ingested, and is used in Table 1.

2.2 Asymmetric Failure-Dependency Projection

In pub/sub middleware, data messages flow from publisher to subscriber ($A \rightarrow B$). However, **structural failure dependency points in the exact opposite direction** ($B \xrightarrow{\text{DEPENDS_ON}} A$): if Publisher A (e.g., radar tracker) fails or produces corrupt messages, Subscriber B (e.g., fire control calculator) is starved or degraded; conversely, if Subscriber B crashes, Publisher A continues unaffected.

Building upon the dependency analysis methodology of Yigit and Buzluca [17], SaaG projects logical DEPENDS_ON edges from structural topology (Table 1).

Table 1: Derived Logical Dependency Edge Projection Rules.

Rule	Type	Derivation Pattern	Weight $w(e)$
1	app_to_app	App/Lib SUB $t \leftarrow$ PUB App/Lib	$\max_t w(t)$
2	app_to_broker	App/Lib PUB/SUB $t \leftarrow$ ROUTES Broker	$\max_t w(t)$
3	node_to_node	Lifted from app_to_app between hosted units	Lifted $\max(w)$
4	node_to_broker	Lifted from app_to_broker for hosted units	Lifted $\max(w)$
5	app_to_lib	App/Lib USES $\rightarrow$ Library	Inherits $w(\text{app})$
6	broker_to_broker	Colocation edge sharing physical Node	Inherits $w(\text{node})$

2.3 Intrinsic QoS & Criticality Weight Propagation

SaaG assigns criticality weights $w(v) \in [0, 1]$ across all entities via an expert-elicited linear weighting of QoS contracts, normalized message size, and topology fan-out:

- (1) **Intrinsic Topic Weight Formula:** For each topic $t \in V_{\text{topic}}$,

$$w(t) = \max(0.01, \beta \cdot \text{QoS_score}(t) + (1 - \beta) \cdot \text{size_norm}(t)) \quad (2)$$

where $\beta = 0.85$, $\text{size_norm}(t) = \min\left(\frac{\log_2(1 + \text{size_kb})}{50}, 1.0\right)$, and:

$$\text{QoS_score}(t) = 0.30 \cdot \text{rel} + 0.40 \cdot \text{dur} + 0.30 \cdot \text{prio} \quad (3)$$

Reliability (RELIABLE=1.0, BEST_EFFORT=0.0), Durability (PERSISTENT=1.0, TRANSIENT_LOCAL=0.5, VOLATILE=0.0), and Transport Priority (CRITICAL=1.0, HIGH=0.66, LOW=0.0).

- (2) **Upward Weight Propagation:** Applications and brokers inherit from the topics $T(x)$ they touch,

$$w(x) = \alpha_x \max_{t \in T(x)} w(t) + (1 - \alpha_x) \text{mean}_{t \in T(x)} w(t) \quad (4)$$

with $\alpha_{\text{app}} = 0.80$ and $\alpha_{\text{broker}} = 0.70$. Libraries amplify with fan-out, $w(l) = \min(1.0, \text{base_w} (1 + \gamma \log_2(1 + DG_{\text{in}}(l))))$ where $\text{base_w} = 0.20$, $\gamma = 0.15$, and $DG_{\text{in}}(l)$ is the in-degree of l ; a node takes the maximum of what it hosts, $w(n) = \max_{v \in \text{RUNS_ON } n} w(v)$.

2.4 Operational Usages: Pipeline Gating and Platform Refactoring

The architectural digital model serves two complementary functions across the naval combat system lifecycle:

1. Pre-Deployment Verification & Pipeline Gating (Forward Mode): During concurrent CI/CD pipeline builds, SaaG instantiates an isolated candidate graph $G_{u'} = (V', E')$ by substituting candidate unit version u' into the baseline platform inventory, guaranteeing baseline immutability. The runner audits $G_{u'}$ against static middleware rules (§3) in < 1.2 s, enforcing fail-closed gating on safety-critical S1/S2 violations.

2. Architectural Improvement & Platform Refactoring (Diagnostic Mode): Beyond release gating, architects query the global baseline model G_{baseline} to drive systemic refactoring: Tarjan's SCC extracts cyclic subgraphs across legacy packages (e.g., WASA allocation against threat evaluation) for decoupling into layered abstractions; topology fan-out combined with intrinsic weights $w(v)$ pinpoints overloaded brokers and high-fanout topics needing replication or decomposition; orphaned topics ($|P(t)| = 0$ or $|S(t)| = 0$) and unreferenced libraries ($DG_{\text{in}}(l) = 0$) expose accumulated configuration debt; and proposed structural changes—migrating applications across hosts, altering a durability contract—can be simulated on the model before production source is touched.

3 Model-Driven Verification Rules & CI/CD Pipeline Gating

SaaG enforces static compliance rules over $G_{u'}$. Table 2 separates rules implemented in the prototype (SaaG-P, •), which are audited statically on descriptors today, from those specified for the target enterprise integration but not yet realized (o).

3.1 Severity Rubric & Alignment with Military Safety Standards (MIL-STD-882E)

MIL-STD-882E [2] classifies the severity of *mishap consequences*, not of static configuration defects, so we *adapt* rather than adopt its four-level scale: each violation is assigned the severity of the worst credible mishap it could contribute to, given the criticality of the channels and hosts it affects. Severity is deliberately decoupled from message transport priority, since a LOW-priority channel may still gate a safety-critical function:

- **S1 (Critical Severity — Catastrophic / Safety Critical):** Flaws causing complete loss of safety-critical functions (e.g., pub/sub RxO mismatch on weapon assignment or primary radar tracking channels, CPU over-allocation on fire control servers).

Table 2: Verification Rules Catalog (• Implemented; ○ Specified).

Policy / Rule	SaaG-P	SaaG-D	Target Middleware Check
Pub/Sub RxO QoS Matching	•	•	Durability & Reliability ($O \geq R$)
Middleware Pre-Conditions	•	•	PARTITION & DOMAIN_ID match
Extended Pub/Sub RxO	○	•	DEADLINE, LIVELINESS, OWNERSHIP
CPU Core Allocation	•	•	Core count $\leq C(v_p)$ per host
CPU Core Non-Overlap	○	•	Pairwise non-overlapping masks
Topic Continuity	•	•	Orphaned topics ($ P = 0 \vee S = 0$)
Tactical Schema Match	•	•	TDL / Link 16 / STANAG match
Tactical Field AST	○	•	Bit-level payload ICD alignment
Circular Dependencies	•	•	Tarjan's SCC on dependencies
Telemetry Drift	○	•	$E_{\text{Observed}} \setminus E_{\text{Designed}}$ via logs

- **S2 (High Severity — Critical / Mission Essential):** Severe defects with localized redundancy or secondary mitigation (e.g., orphaned tactical data link topics, circular dependencies among core WASA engagement planning units).
- **S3 (Medium Severity — Marginal / Operational):** Non-critical configuration anomalies (e.g., unassigned transport priority hints, best-effort auxiliary sensor stream topic leaks).
- **S4 (Low Severity / Informational):** Style and maintenance warnings (e.g., deprecated utility library versions, non-standard topic naming).

3.2 CI/CD Runner Integration: Exit Codes, Delta-Aware Gating, & Waiver Register

SaaG integrates into standard Jenkins and GitLab CI/CD runners via a CLI gate:

$$\text{Exit} = \begin{cases} 0, & \text{PASS: No S1/S2 violations (S3/S4 logged)} \\ 1, & \text{FAIL: S1 (Critical) or S2 (High) detected} \\ 2, & \text{ERROR: Execution / parse failure} \end{cases} \quad (5)$$

Standard CI runners treat non-zero exit codes as build failures. On safety-critical release branches (weapon control, fire authorization, track fusion) SaaG runs **fail-closed**: a tool error (Exit 2) is treated as a gate failure and aborts the pipeline, so a candidate that cannot be analyzed is never allowed through unverified. Non-tactical auxiliary branches may run fail-open, logging Exit 2 as a warning with mandatory audit capture.

3.2.1 Delta-Aware Gating & Waiver Register. In legacy naval codebases carrying pre-existing technical debt, an absolute gate blocks every build if baseline violations exist. SaaG supports *delta-aware*

gating: candidate builds are blocked only if $\text{Violations}(G_{u'}) \setminus \text{Violations}(G_{\text{baseline}}) \neq \emptyset$. Known legacy violations are managed via a Chief Architect-approved *Waiver Register* with cryptographic signatures and expiration dates.

4 Multi-Scale Naval Platform Architectures & Verification Methodology

To assess the practical applicability of SaaG pre-deployment verification in an operational naval combat systems environment, we model the architectural scales and middleware configurations across five major ADVENT platform families (**ADVENT Marti**, **ADVENT Ufuk**, **ADVENT Rota**, **ADVENT Kalyon / Milgem CMS**, and **LHD CMS Flagship**).

4.1 Multi-Domain Combat System Architecture & Platform Profiles

Modern naval combat suites deploy across diverse physical and operational domains, as reflected in the public ADVENT product tree. **Marti** is an airborne C2 fit for maritime patrol aircraft, helicopters, and naval UAVs, handling airborne radar, sonobuoy acoustic processing, and data link relay in a tight single-node footprint ($|V| = 1,498$, $|E| = 4,824$). **Ufuk** is a land-based multi-sensor compilation center integrating coastal radar, AIS, ADS-B, and electro-optical surveillance across 23 server nodes ($|V| = 2,174$, $|E| = 7,142$). **Rota** is the mission management system for USV/UUV/UAV platforms with STANAG 4586 compliance [3] ($|V| = 2,192$, $|E| = 6,980$). **Milgem CMS / Kalyon** is the multi-warfare surface combatant suite (AAW, ASuW, ASW, EW) across 21 console nodes ($|V| = 3,254$, $|E| = 11,460$). **LHD CMS** is the task group flagship fit (e.g., L400 TCG Anadolu) with multi-link gateway centers and joint operations planning over 66 consoles and server chassis ($|V| = 2,775$, $|E| = 9,620$).

Across all platform scales, subsystems rely on **Genieware** publish/subscribe middleware to facilitate real-time sensor-to-shooter coordination and Network Enabled Capability (NEC).

4.2 Continuous Verification & Pipeline Gating Workflow

On every merge request, SaaG runs inside the CI runner (Jenkins, GitLab CI) in four stages. It extracts candidate unit descriptors, IDL schemas, topic bindings, and node affinity masks and builds the isolated candidate multigraph $G_{u'}$; projects inverted failure edges ($B \xrightarrow{\text{DEPENDS_ON}} A$) and hierarchical criticality weights; evaluates the rule catalog over QoS contracts, core pinning bounds, topic connectivity, and SCC cycles; and finally checks new violations against the baseline register and waiver list, returning Exit 0 or Exit 1.

4.3 Evaluation on Representative Architectural Misconfiguration Scenarios

Operational defect records for fielded combat systems are not releasable. We therefore characterize the verification engine by the *classes* of architectural misconfiguration its rule catalog targets, drawn from naval integration experience. These are illustrative fault classes, not measured detection results: Table 3 states, for each

Table 3: Representative naval middleware misconfiguration classes targeted by the rule catalog (● rule implemented in prototype; ○ specified only).

Scen.	Architectural Fault	Sev.	Targeting Rule	St.
A	Pub VOLATILE vs Sub TRANSIENT_LOCAL	S1	Pub/Sub RxO Match	●
B	CPU core mask overlap (cores 2–3)	S1	Core Pinning Non-Overlap	○
C	Orphaned Link 16 forwarding topic	S2	Topic Continuity	●
D	Cyclic dependency (WASA vs threat-eval)	S2	Tarjan SCC	●
E	PARTITION mismatch on sensor gateway	S1	Middleware Pre-Cond.	●

class, whether the targeting rule is implemented in the prototype or specified only. We report no true- or false-positive rates; establishing them requires the configuration data §6 shows we cannot yet obtain.

- **Weapon Channel Durability Mismatch (Scenario A — S1):** Altering the durability of `weapon.assignment.cmd` to VOLATILE while the subscriber requests TRANSIENT_LOCAL. Because the offered durability no longer satisfies the request, the endpoints never match and weapon orders are silently dropped. The prototype’s topic-level RxO check flags the incompatible pair statically.
- **OPCON CPU Core Pinning Contention (Scenario B — S1):** Overlapping CPU core affinity masks between track fusion and 3D tactical display servers on host console processors, causing latency jitter exceeding hard 25 ms deadlines. The prototype checks only aggregate core capacity per host; pairwise mask non-overlap is specified but not implemented, for the data-access reason given in §6.
- **Orphaned Tactical Data Link Topics (Scenario C — S2):** Refactored Link 16 J-series topics left without active subscribers ($|S(t)| = 0$), silently severing cross-platform track feeds.
- **WASA Engagement Planning Dependency Cycles (Scenario D — S2):** Transitive cycles between weapon allocation planners and threat evaluators that induce initialization deadlocks during OPCON start-up.
- **Sensor Gateway PARTITION Mismatch (Scenario E — S1):** A primary and a redundant sensor gateway configured into different middleware partitions, so the redundant path never matches its subscribers and the intended failover is silently absent at commissioning.

5 Computational Performance & Scalability of Digital Model Auditing

5.1 Multi-Scale Naval Platform Benchmarking Setup

Profile provenance. We evaluate latency on five profiles whose *scale parameters*—application, topic, node, and library counts, and the QoS and fan-out distributions used to populate them—were elicited

from architects of the corresponding ADVENT platform lines. The topologies themselves are generated, not exported from fielded systems, for the release-restriction reason given in §6; the profiles therefore reproduce the size and shape of the real deployments but not their contents. They span 184–318 applications, 1,219–2,831 topics, 40–110 libraries, and 1–66 nodes, giving $|V|$ from 1,498 (Martı) to 3,254 (Milgem) and $|E|$ from 4,824 to 11,460. Absolute latencies should be read as characteristic of this scale class rather than as measurements of a specific ship.

Environment: Ubuntu 22.04 LTS (AMD EPYC 7763, 32 GB RAM), Python 3.11 with NetworkX 3.2, in-memory graph construction without external database roundtrips. Each figure is the mean of 500 candidate build evaluations per profile across 5 generator seeds, after 20 warm-up runs.

5.2 Scaling Dynamics & Physical Node Distribution Impact

The benchmark results in Table 4 demonstrate two central insights:

- **Dominance of Graph Construction:** Graph construction accounts for **98.4% to 99.6% of total wall-clock time** across all platforms, while pure graph analysis executes in merely **2 to 14 milliseconds** (0.002–0.014 s).
- **Component Count Does Not Predict Cost:** LHD CMS is measurably slower than Milgem CMS (1.152 s vs 0.760 s) despite having *fewer* components (2,775 vs 3,254), the two differing chiefly in node count (66 vs 21). Construction cost therefore tracks how a deployment is distributed across hosts rather than raw component count, because dependency lifting materializes node-level edges from hosted application pairs. We report this as an observation: we have not isolated the dominant term by profiling, and the per-host $O(k^2)$ colocation bound (k = co-located processes, $k \ll |V|$) is a worst case that does not by itself explain the gap, since LHD averages fewer processes per host than Milgem.

Across all operational configurations, total pipeline gating latency remains ≤ 1.15 s ($< 0.22\%$ of a typical 9-minute automated build).

6 Industrial Lessons: The Data-Acquisition Bottleneck in Architectural Digital Modeling

The primary practical insight from building SaaG inside a naval defense engineering organization is that **the computational cost of graph analysis is negligible, but configuration data acquisition across enterprise and security silos is the true blocker.**

Table 4: Gate Execution Latency Across 5 ADVENT CMS Profiles ($N = 500$).

Platform	Nodes	Comp.	Constr.	Anal.	Total	P95
			(s)	(s)	(s)	(s)
Martı (Air C2)	1	1,498	0.123	0.002	0.125	0.150
Ufuk (Coastal C2)	23	2,174	0.406	0.003	0.410	0.435
Rota (USV/UxV)	1	2,192	0.430	0.005	0.435	0.450
LHD CMS (Flagship)	66	2,775	1.138	0.014	1.152	1.161
Milgem (Frigate)	21	3,254	0.757	0.003	0.760	0.780

Table 5: Specified vs. Implemented Gap Analysis in SaaG Verification Capability for Naval Combat Systems.

Specified Verification Capability	Realized in Prototype	Primary Operational Blocker
1. Endpoint-level pub/sub RxO conformance	Topic-level only	Data: endpoint IDL descriptors held in vendor-siloed toolchains
2. Field-level payload schema alignment	Topic-name match	Data: third-party sensor/weapon ICD parsers not on the build runner
3. Hardware core pinning non-overlap	Host capacity only	Data: OS core-binding scripts reside outside application source repos
4. OS memory & kernel parameter audit	Requirement only	Data: target deployment host profiles unavailable via programmatic CMDB
5. Live architectural drift telemetry	Diff engine spec	Data: operational field-log telemetry repository unbuilt
6. Scored installation suitability model	Exit-code gate	Impl: penalty weights require cross-organization calibration
7. Automated CMDB & topology ingestion	JSON descriptors	Data: enterprise CMDB lacks machine-readable REST export

Deployment status. To be precise about what this paper does and does not report: SaaG runs today as a prototype CLI gate on a GitLab CI runner, consuming hand-curated JSON descriptors exported from application repositories, and enforcing the rules marked • in Table 2. It is not yet wired to an authoritative configuration source, and consequently we report no violation counts, true- or false-positive rates, or defects caught before harbor or sea acceptance testing on a fielded baseline. The evaluation in §5 measures gate latency; the contribution of §6 is the gap analysis explaining why the remaining capabilities are blocked.

6.1 The Anatomy of Configuration Data Silos in Defense Projects

Table 5 shows 6 of the 7 specified capabilities blocked by configuration data silos, of three kinds. Application code lives in Git, while sensor and weapon Interface Control Documents and hardware core pinings live in isolated shipyard commissioning archives under separate release control. QoS contracts are fragmented across XML profiles, C++ pragmas, and startup scripts, with no single authoritative form. And enterprise asset databases are structured for manual inventory audit rather than programmatic query, so a CI runner cannot ask them what a target host looks like.

Recommendation. Organizations introducing architectural digital models into naval combat systems should invest first in declarative, centralized configuration-as-code and machine-readable ICD registries, and only then in graph reasoning engines. The reasoning was never the hard part.

6.2 Limitations

Three limitations bound these results. First, effectiveness is characterized by the fault classes the catalog targets (Table 3), not by measured detection rates on operational baselines; the data needed to establish those rates is exactly what §6 reports as blocked. Second, the criticality weights of §2.3 are expert-elicited and uncalibrated, and we have not run a sensitivity analysis; they order components plausibly for triage but should not be read as validated risk scores. QoS_score also omits DEADLINE, LATENCY_BUDGET, and LIVELINESS, which matter for the hard-deadline channels we motivate. Third, latency is measured on generated topologies at realistic scale (§5.1), so it characterizes a scale class rather than a specific ship.

7 Related Work

Architecture conformance and configuration errors. Murphy et al. [8] pioneered Software Reflexion Models comparing designs against extracted call graphs; Perry and Wolf [9] and de Silva and

Balasubramaniam [10] characterized architectural erosion; Terra and Valente [11] introduced dependency constraint languages; and Yigit and Buzluca [17] formulated graph-based dependency analysis for critical components in pub/sub systems. On the configuration side, Xu and Zhou [12] surveyed configuration error detection, Tang et al. [13] described holistic configuration management at Facebook, and Huang et al. [14] validated cloud configurations declaratively with ConfValley. SaaG carries this line into safety-critical naval pub/sub systems, coupling RxO contract matching to a military severity rubric.

Policy-as-code, middleware diagnostics, and digital twins. Policy engines such as ArchUnit and OPA-Gatekeeper enforce static rules over code and manifests, while DDS and ROS 2 tooling reports QoS incompatibility at runtime [6]—but only once endpoints have initialized in the target environment, which for a combat system means a testbed or a ship. SaaG moves the same class of check left, into the pipeline. Kritzinger et al. [7] established the digital model/shadow/twin taxonomy and Tao et al. [16] reviewed industrial digital twins; we adapt the digital *model* to software architecture as a static, reproducible gating baseline.

8 Conclusion

We presented **SaaG**, an architectural digital model for pre-deployment verification and CI/CD gating in mission-critical Naval Combat Management Systems. It audits candidate releases against pub/sub QoS contracts, core allocations, topic continuity, and dependency cycles inside the pipeline, and gives architects global analytics for diagnosing structural debt in fielded platforms. Across 5 ADVENT profiles spanning single-node airborne units to a 66-node flagship, gating costs ≤ 1.15 s—under 0.22% of an automated build—so graph analysis is not the constraint. The constraint is configuration data acquisition across defense engineering silos, which still blocks 6 of the 7 capabilities our architects specified. Our next step is therefore ingestion rather than analysis: automated ICD harvesters, delta-aware gating across multi-ship task group pipelines, and—once field telemetry exists—the one-way runtime feed that would make this digital model a digital shadow.

References

- [1] ARMERKOM / HAVELSAN. 2022. *ADVENT Combat Management System: Architectural Principles and Operational Capabilities*. Technical Whitepaper, HAVELSAN Inc.
- [2] Department of Defense. 2012. *System Safety (MIL-STD-882E)*. DoD Standard Practice.
- [3] NATO Standardization Office. 2017. *STANAG 4586: Standard Interfaces of UAV Control System (UCS) for NATO UAV Interoperability*, Edition 4.

- [4] NATO Standardization Office. 2020. *STANAG 5516: Tactical Data Exchange - Link 16*, Edition 8.
- [5] NATO Standardization Office. 2021. *STANAG 5522: Tactical Data Exchange - Link 22*, Edition 3.
- [6] Object Management Group (OMG). 2015. *Data Distribution Service (DDS) Specification*, Version 1.4.
- [7] W. Kritzing et al. 2018. Digital Twin in manufacturing: A categorical literature review. *IFAC-PapersOnLine* 51, 11 (2018), 1016–1022.
- [8] G. C. Murphy, D. Notkin, and K. J. Sullivan. 2001. Software reflexion models. *IEEE TSE* 27, 4 (2001), 364–380.
- [9] D. E. Perry and A. L. Wolf. 1992. Foundations for the study of software architecture. *ACM SIGSOFT SEN* 17, 4 (1992), 40–52.
- [10] L. de Silva and D. Balasubramaniam. 2012. Controlling software architecture erosion. *JSS* 85, 1 (2012), 132–151.
- [11] R. Terra and M. T. Valente. 2014. A dependency constraint language. *SPE* 44, 9 (2014), 1073–1094.
- [12] T. Xu and Y. Zhou. 2015. Systems approaches to tackling configuration errors. *ACM CSUR* 47, 4 (2015), 70.
- [13] C. Tang et al. 2015. Holistic configuration management at Facebook. In *ACM SOSP '15*. 328–343.
- [14] P. Huang et al. 2015. ConfValley: Validating system configurations. In *ACM EuroSys '15*. Article 4.
- [15] J. Humble and D. Farley. 2010. *Continuous Delivery*. Addison-Wesley.
- [16] F. Tao et al. 2019. Digital twin in industry: State-of-the-art. *IEEE TII* 15, 4 (2019), 2405–2415.
- [17] I. O. Yigit and F. Buzluca. 2025. A Graph-Based Dependency Analysis Method for Identifying Critical Components in Distributed Publish-Subscribe Systems. In *IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*. <https://doi.org/10.1109/RASSE64831.2025.11315354>